

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ  
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**Дисциплина:** Фронт-энд разработка

**Отчет**

**Лабораторная работа №3**

**Разработка одностраничного веб-приложения (SPA) с  
использованием фреймворка Vue.JS**

**Выполнил:**

**Косарев Илья**

**Группа  
J3210**

**Проверил:**

**Добряков Д. И.**

**Санкт-Петербург**

**2026 г.**

## Задача

Мигрировать ранее разработанное приложение (в рамках ЛР1 и ЛР2) на фреймворк Vue.JS.

Каким требованиям ваше приложение должно соответствовать:

- Должен быть подключён роутер
- Должна быть реализована работа с внешним API (желательно посредством axios)
- Разумное деление на компоненты (продемонстрируйте понимание компонентного подхода)
- Использование composables для выделения повторяющегося функционала в отдельные файлы

## Ход работы

Развернут базовый каркас приложения на базе Vue 3 (Composition API).

Подключен и настроен vue-router для обеспечения навигации без перезагрузки страниц.

Настроен корневой компонент App.vue, выступающий точкой входа.

Разработан компонент-макет AppLayout.vue, включающий в себя глобальные элементы интерфейса (NavBar, SideBar). Для страниц авторизации реализован рендеринг чистого <RouterView> без макета.

```
<script setup>
import NavBar from '../components/NavBar.vue'
import SideBar from '../components/SideBar.vue'
import { RouterView } from 'vue-router'
</script>

<template>
  <NavBar />
  <div class="container-fluid">
    <div class="row">
      <SideBar />
      <main class="col-md-9 col-lg-10" id="profileMain">
        <RouterView />
      </main>
    </div>
  </div>
</template>
```

Рисунок 1 - код MainLayout.vue

```
const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: '/login',
      name: 'login',
      component: LoginView,
      meta: { requiresGuest: true }
    },
    {
      path: '/register',
      name: 'register',
      component: RegisterView,
      meta: { requiresGuest: true }
    },
    {
      path: '/',
      component: MainLayout,
      meta: { requiresAuth: true },
      children: [
        {
          path: '',
          name: 'home',
          component: () => import('../views/HomeView.vue')
        },
        {
          path: 'hub',
          name: 'hub',
          component: () => import('../views/HubView.vue')
        },
        {
          path: 'subscriptions',
          name: 'subscriptions',
          component: () => import('../views/SubscriptionsView.vue')
        },
        {
          path: 'model/:id',
          name: 'model',
          component: () => import('../views/ModelView.vue'),
          props: true
        },
        {
          path: 'dataset/:id',
          name: 'dataset',
          component: () => import('../views/DatasetView.vue'),
          props: true
        }
      ]
    }
  ]
})
```

Рисунок 2 - код router из index.js

Интерфейс разделен на логические и переиспользуемые .vue компоненты, что позволило избавиться от дублирования HTML-кода.

Созданы компоненты: NavBar.vue (навигация), SideBar.vue (меню), ItemCard.vue (универсальная карточка).

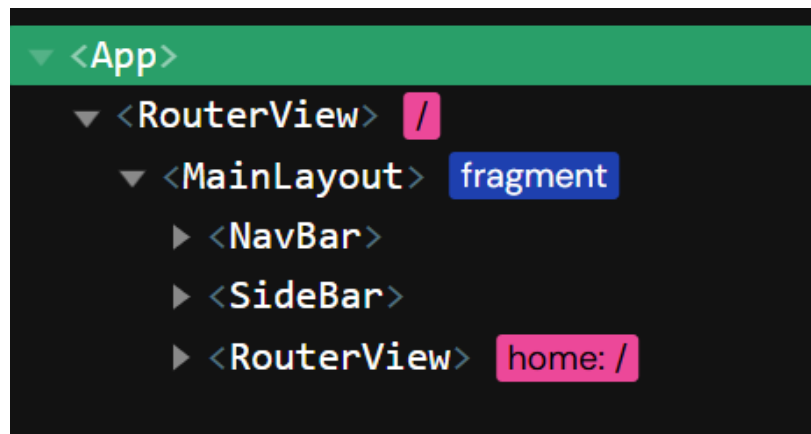


Рисунок 3 - модульная структура на странице личного кабинета

Интегрирован стейт-менеджер Pinia (с использованием синтаксиса Setup Stores).

Создан useAuthStore для централизованного хранения токена и данных профиля пользователя.

Создан useThemeStore для управления логикой переключения светлой/темной темы и синхронизации с localStorage и нативным data-bs-theme.

```
import { ref } from 'vue'
import { defineStore } from 'pinia'

export const useThemeStore = defineStore('theme', () => {
  const currentTheme = ref(localStorage.getItem('theme') || 'auto')

  function setTheme(newTheme) {
    if (['light', 'dark', 'auto'].includes(newTheme)) {
      currentTheme.value = newTheme
      localStorage.setItem('theme', newTheme)
      applyTheme()
    }
  }

  function applyTheme() {
    const darkMedia = "(prefers-color-scheme: dark)"
    const darkLink = document.getElementById('dark-theme-style')

    if (!darkLink) return

    if (currentTheme.value === 'dark') {
      darkLink.media = "all"
      document.documentElement.style.colorScheme = "dark"
      document.documentElement.setAttribute('data-bs-theme', 'dark')
    } else if (currentTheme.value === 'light') {
      darkLink.media = "not all"
      document.documentElement.style.colorScheme = "light"
      document.documentElement.setAttribute('data-bs-theme', 'light')
    } else {
      darkLink.media = darkMedia
      document.documentElement.style.colorScheme = ""
      const prefersDark = window.matchMedia(darkMedia).matches
      document.documentElement.setAttribute('data-bs-theme', prefersDark ? 'dark' : 'light')
    }
  }

  window.matchMedia("(prefers-color-scheme: dark)").addEventListener('change', () => {
    if (currentTheme.value === 'auto') {
      applyTheme()
    }
  })

  return { currentTheme, setTheme, applyTheme }
})
```

Рисунок 4 - код управления стейтом темы

```

import { ref, computed } from 'vue'
import { defineStore } from 'pinia'
import api from '@/services/api'

export const useAuthStore = defineStore('auth', () => {
  const token = ref(localStorage.getItem('jwt_token') || '')
  const user = ref(JSON.parse(localStorage.getItem('user_info') || 'null'))

  const isAuthenticated = computed(() => !!token.value)

  const displayName = computed(() => {
    if (!user.value) return 'Гость'
    const fullName = `${user.value?.firstName || ''} ${user.value?.lastName || ''}`.trim()
    return fullName || user.value?.username || 'Пользователь'
  })

  function saveAuthSession(newToken, newUser) {
    token.value = newToken
    user.value = newUser
    localStorage.setItem('jwt_token', newToken)
    localStorage.setItem('user_info', JSON.stringify(newUser))
  }

  function clearAuthSession() {
    token.value = ''
    user.value = null
    localStorage.removeItem('jwt_token')
    localStorage.removeItem('user_info')
  }

  async function login(email, password) {
    try {
      const response = await api.post('/login', { email, password })
      saveAuthSession(response.data.accessToken, response.data.user)
      return { success: true }
    } catch (error) {
      return { success: false, message: error.response?.data || 'Неверный email или пароль' }
    }
  }

  async function register(userData) {
    try {
      const response = await api.post('/register', userData)
      saveAuthSession(response.data.accessToken, response.data.user)
      return { success: true }
    } catch (error) {
      return { success: false, message: error.response?.data || 'Ошибка при регистрации' }
    }
  }

  function logout() {
    clearAuthSession()
  }

  return {
    token,
    user,
    isAuthenticated,
    displayName,
    login,
    register,
    logout
  }
})

```

Рисунок 5 - код хранения юзер-токена и авторизации

В качестве mock-сервера развернут json-server-auth.

Подключена библиотека axios. В модуле services/api.js создан выделенный инстанс с базовым URL для выполнения HTTP-запросов (GET, PATCH и др.).

```

import axios from 'axios'

const api = axios.create({
  baseURL: 'http://localhost:3000',
  headers: { 'Content-Type': 'application/json' }
})

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('jwt_token')
  if (token) {
    config.headers.Authorization = `Bearer ${token}`
  }
  return config
})

export default api

```

Рисунок 6 - код api.js

Разработан хук useFetch.js для инкапсуляции рутинной работы с API. Хук возвращает реактивные переменные состояния (data, isLoading, error) и метод execute, что значительно сократило объем кода в компонентах-представлениях (Views).

Разработан хук usePageTitle.js, использующий watchEffect для динамического обновления заголовка (document.title) вкладки браузера при загрузке реактивных данных с сервера.

```
import { ref } from 'vue'
import api from '@/services/api'

export function useFetch() {
  const data = ref(null)
  const error = ref('')
  const isLoading = ref(false)

  const execute = async (url, config = {}) => {
    isLoading.value = true
    error.value = ''

    try {
      const response = await api.get(url, config)
      data.value = response.data
      return response.data
    } catch (err) {
      error.value = err.response?.data || 'Ошибка при загрузке данных.'
      throw err
    } finally {
      isLoading.value = false
    }
  }

  return { data, error, isLoading, execute }
}
```

Рисунок 7 - код useFetch.js

```
import { watchEffect, isRef } from 'vue'

export function usePageTitle(title) {
  watchEffect(() => {
    const currentTitle = isRef(title) ? title.value : title

    if (currentTitle) {
      document.title = `${currentTitle} | MLShare`
    } else {
      document.title = 'MLShare'
    }
  })
}
```

Рисунок 8 - код usePageTitle.js

Нативные скрипты Bootstrap (управляющие вкладками) заменены на реактивное управление состоянием Vue. Переключение контента реализовано через директивы v-show и v-if.

## **Вывод**

В ходе выполнения лабораторной работы была успешно проведена миграция статического многостраничного веб-приложения на современный фреймворк Vue.js (Composition API). Проект был преобразован в полноценное SPA (Single Page Application) с помощью Vue Router, что обеспечило плавную маршрутизацию без перезагрузки страниц.

В результате получено легко масштабируемое, реактивное и удобное в поддержке веб-приложение, полностью удовлетворяющее современным стандартам Frontend-разработки.